Online Partial Computation Offloading Optimization in Wireless Powered Mobile Edge Computing Network

Lu Sun, *Member, IEEE*, Rina Liang, Liangtian Wan, *Member, IEEE*, Kaihui Liu, *Member, IEEE*, Zhaolong Ning, *Senior Member, IEEE*, and Jie Wang, *Senior Member, IEEE*

***Abstract*— Computation offloading is one of the most crucial issues of wireless powered mobile edge computing (WP-MEC) network, which decides how to offload the tasks of wireless devices (WDs) to the access point (AP) for further intensive computation. However, in practical application, efficient computation offloading and resource scheduling are challenging with online factors, i.e., the time-varying channel conditions and random task arrivals. Compared with binary offloading decisions, partial offloading decisions can be flexibly adjusted according to the characteristics of the task. The proliferation of feasible partial offloading schemes also brings challenges to the optimization of computational offloading policies. To resolve the above problems, this paper proposes an online partial computation offloading scheme based on Lyapunov optimization and deep reinforcement learning (LyDROP) in a WP-MEC network, where the wireless power transfer (WPT) duration time, partial offloading decisions and transmission time allocation are jointly optimized. The objective is to maximize the weighted sum computation rate of all wireless devices while reducing the data queue backlog as much as possible. Firstly, Lyapunov optimization is used to decouple the stochastic online problem into a series of per-frame deterministic problems. Subsequently, combination of model-based optimization and model-free machine learning algorithms is adopted to perform extended simulation experiments. The performance of LyDROP is validated utilizing a real-world dataset of Melbourne in Australia. Finally, in comparison with several baseline or state-of-the-art algorithms, including Lyapunov-optimization-based coordinate decent (LyCD), Lyapunov-optimization-based randomly partial computation offloading (LyRPO) and Myopic algorithm, the superiority of LyDROP algorithm is demonstrated.**

***Index Terms*— Online computation offloading, mobile edge computing, wireless power transfer, Lyapunov optimization, deep reinforcement learning.**

Received 4 April 2025; revised 16 July 2025 and 21 August 2025; accepted 16 September 2025. Date of publication 22 September 2025; date of current version 29 December 2025. This work is supported by National Natural Science Foundation of China (62571080), Fundamental Research Funds for the Central Universities (3132025248), by Guangdong Basic and Applied Basic Research Foundation under Grant 2022A1515140074, by the Key Laboratory of Big Data Intelligent Computing Funds for Chongqing University of Posts and Telecommunications (BDIC-2023-A-003), by the Science and Technology Research Program for Chongqing Municipal Education Commission KJZD-M202200601, by the Science and Technology Program of Liaoning Province under grant 2023JH2/101300199, and Dalian Science and Technology Innovation Foundation under grant 2022JJ12WZ057. The associate editor coordinating the review of this article and approving it for publication was N. Zorba. *(Corresponding author: Liangtian Wan.)*

Lu Sun, Rina Liang, and Jie Wang are with the Department of Communication Engineering, Institute of Information Science Technology, Dalian Maritime University, Dalian 116026, China (e-mail: sunlu@dlmu.edu.cn; rinaliang@dlmu.edu.cn; wang_jie@dlmu.edu.cn).

Liangtian Wan is with the Key Laboratory for Ubiquitous Network and Service Software of Liaoning Province, School of Software, Dalian University of Technology, Dalian 116620, China (e-mail: wanliangtian@dlut.edu.cn).

Kaihui Liu is with the School of Electronic Engineering, Dongguan University of Technology, Dongguan 523820, China (e-mail: kaihuiL@outlook.com).

Zhaolong Ning is with the School of Communication and Information Engineering, Chongqing University of Posts and Telecommunications, Chongqing 400065, China (e-mail: z.ning@ieee.org).

Digital Object Identifier 10.1109/TCCN.2025.3612741

I. Introduction

RECENTLY, research on the fusion of mobile edge computing (MEC) and wireless power transfer (WPT) has drawn considerable attention, because it has great potential in enhancing the device computation capacity and prolonging the device lifetime [1], [2]. MEC sinks the services and functions from the data center to the edge of the network, offering rich computational and storage resources for augmented reality (AR) and virtual reality (VR) applications [3]. MEC facilitates the caching of audio and video content, sending it to the user or quickly simulating the 3D dynamic scene for real-time interaction. There are a large number of end users in the internet of vehicles scenario, and a variety of services exist accordingly. Vehicles with spare resources can serve as MEC nodes and handle high priority emergencies as well as computationally intensive services, thereby enhancing the user experience [4]. Owing to the limited battery power of terminal devices, the computational performance of MEC is limited. Generally speaking, the methods of energy harvesting are characterized by two principal categories: renewable energy harvesting and WPT [5]. However, the unpredictability of the former energy generation method may limit the task processing capability of WDs. WPT offers dependable and stable power supply that makes the realization of many innovative applications possible, paving the way of novel services advancements. WPT technology has been applied in residential settings to charge smart home devices [6]. WPT can power wearable devices or telemedicine devices, providing convenience to users and simultaneously enhancing medical efficiency.

In a WP-MEC network, the energy transmitter emits a radio frequency (RF) signal to WDs, from which the WDs obtain energy to transmit information. As a consequence of the controllability of WPT, the interaction between the user's computing energy requirements and the wireless power supply of the edge nodes can be effectively adjusted and balanced [7]. Taking into account the power consumption profile of computing and the feasibility of wireless power

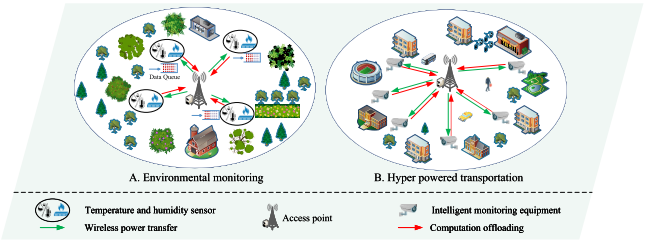


Fig. 1. Application scenarios of WP-MEC network.

transfer in a long distance, Wang et al. studied the WP-MEC network within a 1 km × 1 km area of the Manhattan city map, and validated the effectiveness of the proposed algorithm [8]. Based on the above advantages, Fig. 1 shows the scenario in which WP-MEC network can be flexibly deployed. WP-MEC network enables low-power WDs to interact and compute efficiently in Internet of things (IoTs) applications. For example, in environmental sensing applications, WDs depend on the collected RF energy to process environmental data (such as air pressure, visibility, wind, etc.) locally or transmit to the access point (AP) for data analysis and computational operations. In addition, the application of WP-MEC in the hyper powered transportation system can provide wireless charging services for intelligent road monitoring equipment. Integrating WP-MEC network within the hyper powered transportation system can not only diminish the dependence on the traditional power grid, but also realize the user needs of transmitting high-definition video.

As a prominent research domain in WP-MEC system, computation offloading is recognized an essential technological strategy, aimed at overcoming many limitation of WDs [9]. Inefficient offloading strategies (such as unreasonable task allocation) can lead to insufficient energy collected by WDs to complete computing tasks, resulting in task data queue backlog. For scenarios with high real-time requirements such as AR or VR, it may cause scene lag and affect the user experience. In addition, in dense deployment scenarios, inefficient offloading strategies may cause load imbalance of edge servers, that is, some servers are overloaded while others are idle, seriously affecting the efficiency of resource utilization. In recent years, some studies have investigated the joint WPT, communication, and computing resource allocation of WP-MEC systems in various settings [10], [11], [12], [13], [14]. He et al. considered a cooperative three-node system that integrated a source node, hybrid access point equipped with MEC server and a helper node to enhance the system performance [10]. Different from the problem of energy efficiency explored in [10], Li et al. proposed the user's energy consumption problem in the same scenario and introduced non-orthogonal multiple access (NOMA) technology, aiming to overcome the impact of double far-near [11]. Although Zeng et al. considered the same scenario as [11] and similarly leveraged NOMA technology, Zeng et al. focused on the energy consumption associated with AP transmission rather than the user's energy consumption [12]. Zhang et al. conducted an in-depth study on terminal pairing problem in WP-MEC systems and proposed an innovative bilevel optimization method to solve the complex issue of multi-dimensional resource allocation [13]. Through this method, the energy consumption was reduced and the efficiency of cooperative computation was enhanced. Bi and Zhang investigated task offloading as well as resource allocation in a multi-user WP-MEC network and adopted method of coordinate descent method together with alternating direction multiplier method (ADMM) respectively to solve the computational offloading problems of WDs [14]. Furthermore, the author discussed the corresponding application scenarios of the above two algorithms.

While the aforementioned studies have made valuable contributions to the WP-MEC system, it is essential to note that these efforts may not fully address the emerging challenges presented by dynamic channel and task environments in real-world systems. Time-varying channel conditions lead to dynamic changes in the energy collected by the terminal device, which in turn affects its own computing performance. The randomness of the task quantity arriving at the terminal device increases the complexity of the system. Furthermore, the above-mentioned literature fails to fully combine the randomness of task arrival with the computing power of the terminal device itself. Making full use of the computing power of terminal devices is the key to designing an efficient offloading strategy, and intelligent task allocation needs to be achieved between local computing and edge computing. Accordingly, online factors, i.e., dynamic channel and random task in WP-MEC scenarios are considered in this paper, and the computing power of local devices is taken into account in this paper, which is more in line with the needs of actual scenarios. The design of corresponding online offloading algorithms is confronted with two significant challenges. Firstly, obtaining the best offloading solution is difficult when the future system parameters are uncertain [15]. When the system parameters change, traditional numerical methods such as the coordinate descent (CD) method need to frequently re-solve complex computing offloading problems. In particular, it is difficult for the above methods to obtain the best offloading scheme in a short time when the parameter scale is large. Secondly, owing to the uncertainty of future system information, the implementation of long-term constraints of the system poses a significant challenge for online offloading decisions [16]. To tackle the aforementioned challenges, this paper combines Lyapunov optimization with deep reinforcement learning (DRL). Lyapunov optimization can address long-term time-varying problems by providing limited performance guarantees under uncertain conditions [9], while DRL can be applied in addressing online computation offloading challenges in dynamically changing environments [17], [18], [19]. DRL interacts with the environment (such as channel states and task data queues), utilizes deep neural network (DNN) to extract features from time-varying factors, updates the optimal offloading strategy online, and achieves efficient resource allocation simultaneously.

A. Contributions

The major contributions of the paper are outlined as follows:

- We consider an online multiuser WP-MEC framework with time-varying channel conditions and random task arrivals. And we formulate an online optimization problem of maximizing the weighted sum computation rate of all WDs while reducing the data queue backlog as much as possible. It requires a comprehensive coordination of

the WPT duration time, partial offloading decisions and transmission time allocation.

- We put forward and design an online partial computation offloading algorithm utilizing Lyapunov optimization algorithm and deep reinforcement learning (LyDROP). The Lyapunov optimization is introduced to transform stochastic online problems into per-frame deterministic problems. Subsequently, the per-frame deterministic problems are solved by combining the complementary strengths of model-based optimization and DRL.
- We leverage the actor-critic structure with the aim of resolving the per-frame problems. In the Actor section, we propose partial offloading noise-order-preserving quantization method, which can significantly reduce the backlog of task data queues. In the Critic part, the Lagrange duality method with lower computational complexity is used to solve the allocation of wireless energy transmission time and offloading time.
- We conduct a series of simulation experiments and undertake theoretical analysis of the results. We confirm the superiority of LyDROP by comparing with multiple existing solutions. Meanwhile, for the purpose of validating the effectiveness of the designed algorithm, performance evaluation is conducted utilizing a real-world dataset of Melbourne in Australia.

B. Organization of the Paper

The remaining of the paper is structured in the following. Section II introduces the related works concerning online computing offloading. Section III presents the system model and optimization problem. Subsequently, the LyDROP algorithm is proposed and designed in Section IV. Section V introduces simulation results. Finally, the paper is concluded and future research directions are presented in Section VI.

II. Related Work

There exists limited work on long-term system dynamics, particularly with respect to dynamic channel or random task for WP-MEC networks [17], [20], [21], [22]. Huang et al. formulated an online offloading framework that leveraged DRL to achieve the maximization of computation rates for WDs [20]. Zheng et al. addressed the challenge of minimizing total computation delay and optimized the WPT duration together with transmission time allocation using worst-WD-adjusting algorithm [21]. Hu et al. dedicated their investigation to a reinforcement learning-based strategy under time-varying channel conditions, which aimed to diminish latency and energy consumption in multi-WDs dynamic MEC environments [17]. Similar to [20], Zhang et al. concentrated their research efforts on computation rate in WP-MEC network as well [22]. One of the differences in algorithm was that [20], and [22] used DNN for the purpose of obtaining offloading strategies and determining WPT time, respectively. However, these methods only consider time-varying channel conditions and do not take into account of time-varying data arrivals. In [17], [20], and [21], binary computation offloading is assumed.

In general, the strategies for computation offloading are primarily categorized into two methods, namely binary offloading [8], [14], [17], [20], [21], [23], [24] and partial offloading [5], [25], [26], [27], [28], [29]. Binary offloading demands task to be performed locally in its entirety or, conversely, offloaded entirely to the edge server for remote processing [23]. Bi et al. investigated a MEC framework characterized by dynamic channel and random task, without introducing WPT into the network [23]. The author proposed an innovative algorithm that effectively combined the strengths of Lyapunov optimization and DRL. DRL learns the control strategy from the raw data, expands the state space and possible action space of the system, and receives rewards. Since the strengths of both deep learning and reinforcement learning are incorporated in DRL, the combination of MEC and DRL can better match the task characteristics and the real-time state of the system. Chen et al. conducted an analysis of energy consumption for online WP-MEC network and introduced an enhanced two-stage deep Q-network (TS-DQN) framework to effectively manage and reduce the consumption [24]. Wang et al. investigated WP-MEC network containing multiple APs and multiple mobile users, and then proposed a novel distributed online learning algorithm [8]. In addition, some articles assume that sensors do not have local computing capabilities and adopt the full computation offloading policy [30], [31], [32]. Under the aforementioned strategy, all task data collected by WDs is offloaded to the base stations (BSs) equipped with MEC server, where it is subsequently processed by the server. For the purpose of maximizing system utility, Wu et al. proposed an online optimization algorithm, which leveraged Lyapunov and convex optimization techniques and maintained the asymptotic optimality [30]. With the same objective function as [30], Wu et al. investigated an online algorithm using Lyapunov optimization in WP-MEC based heterogeneous industrial Internet of things (IIoTs) system. The algorithm decoupled the initial issue into a series of deterministic subproblems, and then solved it using convex optimization technique [31]. Deng et al. introduced the concepts of task and energy queue to address the throughput maximization problem of WP-MEC system, which was composed of a number of BSs and mobile devices (MDs) [32]. In the dynamic WP-MEC network, binary offloading only allows all tasks to be computed locally or completely offloaded. It is difficult to quickly adapt to the dynamically changing computing requirements, which may lead to resource waste and performance bottlenecks.

Compared with binary offloading, partial offloading has significant advantages in terms of flexibility, resource utilization efficiency, etc. Partial offloading divides each task into many subtasks [26]. Partial offloading decisions can flexibly adjust the offloading ratio based on the availability of device resources, task requirements, and channel conditions. Moreover, the task data of the local and edge servers can be processed in parallel, shortening the task completion time. In real-world scenarios, such as in smart healthcare, the sensitive parts of task data can be processed locally, and only the non-critical parts can be offloaded. Compared with binary offloading, partial offloading simultaneously meets the requirements of real-time data processing and privacy protection. In autonomous driving,

subtasks with high latency tolerance can be unloaded and emergency subtasks can be processed locally, which simultaneously satisfies real-time performance and computational efficiency.

Using the Lyapunov and convex optimization techniques, Mao et al. conducted an in-depth analysis of the latency and energy efficiency tradeoffs in WP-MEC network, which included an AP, a power beacon and users [5]. Javadpour [33] focused on the optimization techniques of resource management in the edge computing environment. Javadpour [34] and Javadpour [35] respectively explored the task scheduling problem from different dimensions: The authors of [34] innovatively proposed a dual-algorithm solution of sparse functions and genetic algorithms, effectively enhancing the efficiency of task allocation; The authors of [35] focused on the task execution period and proposes an algorithm based on task priority. To solve the complex problems of action spaces and the complexities associated with device coordination, Zhang et al. introduced two DRL algorithms in MEC systems with energy harvesting devices [29]. In order to achieve the energy transmitter's energy transmission minimization as well as ensuring the prescribed computation deadlines of tasks, Bolourian et al. conducted a comprehensive study on three-tier WP-MEC system, which integrated WDs, edge servers and cloud servers [26]. And the author assumed that each task of the device could be divided into a series of subtasks through the approach of program partitioning. Wang et al. investigated a unmanned aerial vehicles (UAV)-assisted WP-MEC network that could facilitate applications of human-centric metaverse, aiming to maximize the system's average computation efficiency from a long-term perspective [27]. Hu et al. investigated problems such as power control associated with UAV trajectory scheduling in UAV-assisted WP-MEC network by combining randomness of task arrival and variability of wireless channel environment [25]. The method proposed by the authors was incapable of leveraging historical data to gain experience. Consequently, in a rapidly changing edge environment, complex computational offloading problems must be frequently re-evaluated and resolved. Owing to the limited battery capacity, UAVs are capable of only brief operational and flight durations, typically spanning from a few tens of minutes up to an hour [36]. Guo et al. studied the issue of energy minimization in MEC system using partial computation offloading schemes, taking the task discard rate as a key performance metric [28]. Similar with [23], Lyapunov optimization technology was also utilized. Employing Lyapunov optimization, a multi-stage random problem is decoupled into per-stage deterministic subproblems, which theoretically guarantees long-term system stability. The authors of [23] and [28] both consider time variability of channel conditions and randomness of task arrivals, but do not take the WP-MEC scenario into consideration.

In summary, different from all existing work, this paper introduces an innovative online multiuser WP-MEC framework with dynamic channel and random task arrival. The task arrivals for all WDs follow exponential distribution [23]. And then we design LyDROP algorithm in solving the online partial computation offloading optimization problems.

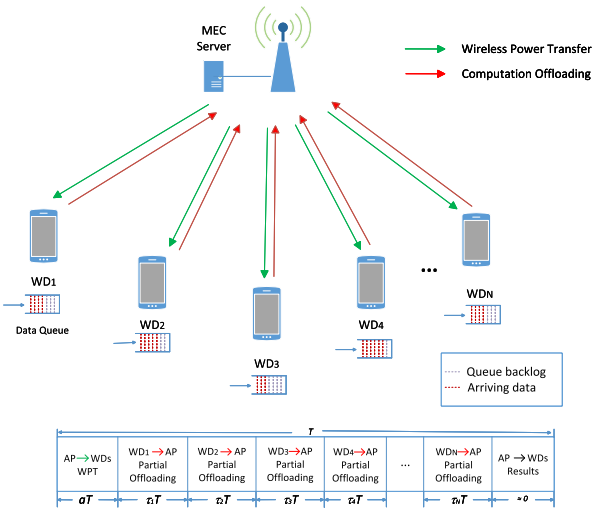


Fig. 2. An example of online multi-user WP-MEC system.

III. System Model and Problem Formulation

A. System Model

As illustrated in Fig. 2, the scenario we consider involves an AP that provides computational assistance to WDs, represented by a set $\mathcal{N} = \{1, 2, \ldots, N\}$. Each device is equipped with an antenna, a rechargeable battery which is designed to collect energy and provide power for WDs' operation. The system time is partitioned into a series of consecutive, equal-length time frames, each represented by T, and these time frames are defined to be less than the channel coherence time. Assume that employing a frame-based TDMA scheme, WPT and computation offloading occur on the same frequency band. When a time frame begins, all WDs simultaneously capture the energy transmitted by the AP. Let aT represent the amount of time spent on WPT, where $a \in [0,1]$. Let $\tau_i T$ indicate the transmission time for the ith WD offloading, where $\tau_i \in [0,1]$.

Supposed that the channel maintains reciprocity between the downlink and uplink, static for the duration of a single time frame, but potentially changing with respect to different time frames [20], [30]. Let h_i^t represent the channel gain between the ith WD and AP at the tth time. Let $\sigma \in (0, 1)$ stand for energy harvesting efficiency. Let P_a stand for AP's transmitted power. Let E_i represent the energy obtained by the ith WD, and E_i can be written as

$$E_i = \sigma P_a h_i^t a T, \quad \forall i \in \mathcal{N}. \tag{1}$$

The WD can select whether to offload its tasks to the AP, including in what proportion, i.e., partial offloading. Let $\delta_i \in [0, 1]$ indicate the proportion of partial offloading for the ith WD, that is, δ_i of the energy is used for computation offloading, and $(1 - \delta_i)$ of the energy is applied to compute locally. Let R_i^t represent the amount of raw task data that reached the ith WD at the tth time. Assuming R_i^t follows a second-order moment bounded i.i.d. distribution [23], i.e., $\mathrm{E}[(R_i^t)^2] = \rho_i < \infty$, for $i = 1, \cdots, N$. Suppose ρ_i is known to be estimated from past observations. During the tth time frame, assume that the ith WD handles D_i^t bit data, subsequently generating a computational result at the end of the time frame. For the convenience of presentation, the notations used are provided in Table I.

TABLE I
LIST OF NOTATIONS

Symbol	Description
i	The index of WDs
N	The number of WDs
T	The length of a time frame
B	The bandwidth of the wireless channel
h_i^t	The wireless channel gain between the ith WD and the AP at the tth time
a	The proportion of WPT duration
τ_i	The proportion of task offloading time for the ith WD
E_i	The amount of energy harvested by the ith WD
P_a	The transmitted power of the AP
σ	The energy harvesting efficiency
δ_i	The proportion of partial offloading for the ith WD
R_i^t	The raw task data that reached the ith WD at the tth time
f_i	The processor's computing speed of the ith WD
t_i	The computation time of the ith WD
φ	The number of cycles needed to process one bit of task data
k_i	The energy efficiency coefficient
χ	The communication overhead
N_0	The receiver noise power
τ_m	The task computation time of the AP
ω_i	The weight of the ith WD
$D_{i,L}$	The amount of data processed locally
$r_{i,L}$	The local computing rate
$D_{i,O}$	The amount of data transmitted in the offloading mode
$r_{i,O}$	The transmission rate in the offloading mode
D_i^t	The total amount of data processed by the ith WD at the tth time
r_i^t	The total computing rate of the ith WD at the tth time
$Q_i(t)$	The length of the data queue for the ith WD at the tth time

In local computing mode, WD can perform both energy acquisition and computation tasks. Let f_i represent the speed of the processor (cycles per second) for the ith WD. Let t_i represent the computation time of the ith WD, where $0 \le t_i \le T$. Let $\varphi > 0$ represent the number of cycle required to deal with one bit of task data. The number of bits processed by the ith WD in each time frame is $f_i t_i/\varphi$. The power consumption of the ith WD is $k_i f_i^3$, at the same time the corresponding energy consumption is $k_i f_i^3 t_i$, where k_i indicates the energy efficiency coefficient [37]. Therefore, $k_i f_i^3 t_i \le (1-\delta_i)E_i$. The collected energy will be used up in one time frame for a better calculated rate, i.e., $t_i^* = T$. Then, $f_i^* = (\frac{(1-\delta_i)E_i}{k_i T})^{\frac{1}{3}}$. The amount of data processed locally is

$$D_{i,L} = \frac{f_i^* t_i^*}{\varphi} = (\frac{(1-\delta_i)\sigma P_a h_i^t a}{k_i})^{\frac{1}{3}} \frac{T}{\varphi}, \quad \forall i \in \mathcal{N}. \tag{2}$$

The local computing rate is

$$r_{i,L} = \frac{D_{i,L}}{T} = \frac{1}{\varphi}(\frac{(1-\delta_i)\sigma P_a h_i^t a}{k_i})^{\frac{1}{3}}, \quad \forall i \in \mathcal{N}. \tag{3}$$

If the ith WD offloads the task data to the AP, the transmission power is $P_i = \frac{\delta_i E_i}{\tau_i T}$. The amount of data that can be transmitted is

$$\begin{aligned} D_{i,O} &= \frac{B\tau_i T}{\chi}\log_2(1+\frac{P_i h_i^t}{N_0}) \\ &= \frac{B\tau_i T}{\chi}\log_2(1+\frac{\delta_i \sigma P_a (h_i^t)^2 a}{\tau_i N_0}), \quad \forall i \in \mathcal{N}, \end{aligned} \tag{4}$$

where B indicates the transmission bandwidth, χ represents communication overhead, and N_0 represents the noise power.

In computation offloading mode, the computing rate is

$$r_{i,O} = \frac{D_{i,O}}{T} = \frac{B\tau_i}{\chi}\log_2(1+\frac{\delta_i \sigma P_a (h_i^t)^2 a}{\tau_i N_0}), \quad \forall i \in \mathcal{N}. \tag{5}$$

Let's define $D_i^t \triangleq D_{i,L} + D_{i,O}$, so the computing rate is $r_i^t = \frac{D_i^t}{T}$, that is to say,

$$r_i^t = \frac{1}{\varphi}(\frac{(1-\delta_i)\sigma P_a h_i^t a}{k_i})^{\frac{1}{3}} + \frac{B\tau_i}{\chi}\log_2(1+\frac{\delta_i \sigma P_a (h_i^t)^2 a}{\tau_i N_0}). \tag{6}$$

The task computation time of the AP is

$$\tau_m = \frac{\varphi D_{i,O}}{f_0}, \tag{7}$$

where f_0 represents computational speed of AP. The AP computation latency depends on the computation of the task, the AP computation resource allocation, and the possible queue waiting time. The edge server possesses significantly greater computational capabilities compared to WDs, so the execution time is negligible [14]. The latency associated with feedback is also considered negligible, given that the result size is substantially smaller in comparison to the original data size [28].

The above assumption that the AP computing time and feedback latency can be ignored is applicable to lightweight tasks, medium and low load, and low-latency network environments. In actual deployment, if the complexity or concurrency of tasks increases significantly, the scheduling of AP computing resources needs to be considered. In scenarios where the complexity or concurrency of tasks increases significantly, if computation time at the AP is ignored, it may lead to insufficient computing resources and cause task stacking at AP side.

The feedback latency is

$$\tau_{result} = \frac{D_{result}}{r_{down}}, \tag{8}$$

where D_{result} represents the size of the computation result, and r_{down} represents the downlink transmission rate. In scenarios where the complexity or concurrency of tasks increases significantly, ignoring the feedback latency of computation results may lead to the failure of task scheduling strategies, that is, mistakenly believing that some tasks are suitable for offloading, but in fact, the return delay is too high.

Therefore, the time allocation in each time frame satisfies

$$a + \sum_{i=1}^{N} \tau_i \le 1, \forall i \in \mathcal{N}. \tag{9}$$

If the energy harvested is insufficient to meet the device's consumption, that is, the amount of data arriving at the WD in the current time frame cannot be fully processed, a data backlog will arise. Let $Q_i(t) \ge 0$ indicate the length of the data queue for the ith WD, i.e., data that is queued in the data buffer but has not yet been processed at the current time. Assume that the data queue is in first in first out (FIFO) mode. So the data queue's length at the next time is:

$$Q_i(t+1) = Q_i(t) - D_i^t + R_i^t. \tag{10}$$

Set $D_i^t \leq Q_i(t)$. The discrete queue $Q_i(t)$ is characterized as strongly stable if the time average queue length meets the following conditions: $\lim_{k\to\infty} \frac{1}{k} \sum_{t=1}^{k} \mathbb{E}[Q_i(t)] < \infty$.

B. Problem Formulation

In light of the system model previously described, the goal is to maximize the weighted computation rate for all WDs, which is related to WPT duration, offloading decisions as well as transmission time allocation. Let $\delta = \{\delta_i | i \in \mathcal{N}\}$ and $\tau = \{\tau_i | i \in \mathcal{N}\}$ In summary, the formulated problem (P1) is described as follows:

$$\underset{\delta,\tau,a}{\text{maximize}} \quad \lim_{k\to\infty} \frac{1}{k} \sum_{t=1}^{k} \sum_{i=1}^{N} \omega_i r_i^t \tag{11a}$$

$$\text{subject to} \quad a + \sum_{i=1}^{N} \tau_i \leq 1, \forall i \in \mathcal{N}, \tag{11b}$$

$$D_i^t \leq Q_i(t), \forall i \in \mathcal{N}, \tag{11c}$$

$$\lim_{k\to\infty} \frac{1}{k} \sum_{t=1}^{k} \mathbb{E}[Q_i(t)] < \infty, \tag{11d}$$

$$0 \leq a \leq 1, 0 \leq \tau_i \leq 1, 0 \leq \delta_i \leq 1. \tag{11e}$$

where ω_i in formula (11a) represents the weight of the ith WD. (11b) represents the time allocation constraints of each time frame. (11c) indicates that the amount of task data processed by the ith WD cannot exceed its own data queue backlog. (11d) stands for the ith WD task data queue stability constraint. (11e) represents WPT time proportion constraint, transmission time proportion constraint and partial offloading proportion constraint, respectively.

IV. Algorithm Description

In this section, we propose the LyDROP algorithm. The framework of the LyDROP is illustrated in Fig. 3. It contains four modules, namely the Actor module, the Critic module, the policy update module and the task data queue update module. Specifically, the Actor module contains deep neural networks and noise order-preserving quantization strategies. The Critic module contains Lagrange duality for solving wireless energy transfer time and offloading time allocation (a, τ_i). The strategy update module is the neural network memory pool, which stores the empirical data composed of the environmental parameters of the model and the optimal part offloading decision in the neural network memory pool. This mechanism enables policy-based optimization methods to sample from historical experience and achieve single-step optimization, rather than merely relying on the complete trajectory after the end of the round for training. The data queue update module refers to the process of performing joint actions and completing the update of the wireless device task data queue after finding the optimal offloading decision and resource allocation strategy.

A. Lyapunov Optimization Method

The Lyapunov function quantifies the metrics of all constraints at each moment, evaluating each conditional constraint in the form of a queue sum of squares. Let $\Theta(t) = \{Q_i(t)\}_{i=1}^{N}$. The Lyapunov function is

$$L(\Theta(t)) = \frac{1}{2}\Theta(t)^2 = \frac{1}{2}\sum_{i=1}^{N} Q_i(t)^2. \tag{12}$$

The Lyapunov function represents the degree of queue congestion in the system. A larger value indicates that at least one queue is congested, and the longer the user waits for processing.

The Lyapunov drift is used to weigh the choice of resource allocation strategies. According to the Law of Telescoping Sums, $\sum_{\tau=0}^{t-1} [f(\tau+1) - f(\tau)] = f(t) - f(0)$, the main idea of Lyapunov drift argument is that the final value of the function can be controlled by controlling the change of the function at each step [38]. If the control strategy is optimized to minimize the amount of growth per slot, then the queue backlog will continue to favor lower congestion levels, thus ensuring the overall network's stability. The condition Lyapunov drift is

$$\triangle L(\Theta(t)) = \mathbb{E}\{L(\Theta(t+1)) - L(\Theta(t)) | \Theta(t)\}. \tag{13}$$

At present, we only consider using queue and Lyapunov drift to satisfy the mean time constraint, and the objective function is not jointly considered. So map the objective function to the appropriate penalty function. The Lyapunov drift-plus-penalty function is

$$\triangle_V L(\Theta(t)) = \triangle L(\Theta(t)) - \mathbf{V} \sum_{i=1}^{N} \mathbb{E}\{\omega_i r_i^t | \Theta(t)\}, \tag{14}$$

where $V > 0$ is a control parameter of scaling penalty. We drive an upper bound of $\triangle_V L(\Theta(t))$ as described in *Lemma 1*.

Proof: As described in *Lemma 1*, the corresponding proof is furnished in the Appendix. ■

Lemma 1: The Lyapunov drift-plus-penalty is defined by the equation presented below:

$$\begin{aligned} \triangle_V L(\Theta(t)) \leq C + \mathbb{E}\{Q_i(t)(R_i^t - D_i^t) | \Theta(t)\} \\ -V\mathbb{E}\{\omega_i r_i^t | \Theta(t)\}, \end{aligned} \tag{15}$$

where $C = \sum_{i=1}^{N} ((R_{i,max}^t)^2 + (D_{i,max}^t)^2)$, $R_{i,max}^t$ and $D_{i,max}^t$ are the upper bound of R_i^t and D_i^t, respectively.

Utilizing the Lyapunov optimization theory, the further operation involves expression minimization of the right side of inequality (15), thereby deriving the optimal solution. In the tth time frame, the opportunistic expectation minimization technique is applied [38]. By disregarding the constant terms, we are enabled to resolve problem (P2) by maximizing the following objective function:

$$\underset{\delta,\tau,a}{\text{maximize}} \quad \sum_{i=1}^{N} (Q_i(t) + V\omega_i) r_i^t \tag{16a}$$

$$\text{subject to} \quad a + \sum_{i=1}^{N} \tau_i \leq 1, \forall i \in \mathcal{N}, \tag{16b}$$

$$D_i^t \leq Q_i(t), \forall i \in \mathcal{N}, \tag{16c}$$

$$0 \leq a \leq 1, 0 \leq \tau_i \leq 1, 0 \leq \delta_i \leq 1. \tag{16d}$$

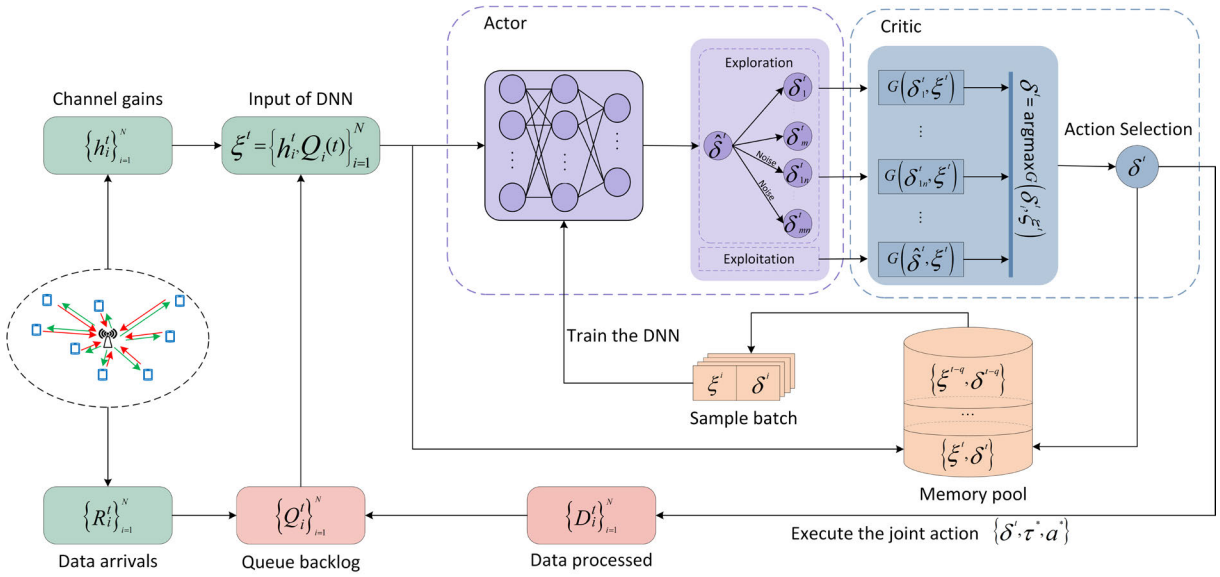


Fig. 3. An illustration of the proposed LyDROP framework.

It is observed that (a, τ_i) is jointly concave, and once δ_i is determined, the above transforms into a convex problem. Model-based optimization method as well as model-free machine learning algorithm are used to solve (a, τ_i) and δ_i, respectively.

B. Partial Offloading Decision Based on DRL

The traditional actor-critic model is composed of two neural networks, and parameters are updated in a continuous manner. However, the correlation between the pre-update parameters and the post-update parameters may causes the neural network to adopt a limited perspective on the problem. To avoid the above problem, we combine the strengths of model-based optimization and model-free machine learning algorithms. The details are as follows:

1) Actor model: The channel gain $\{h_i^t\}_{i=1}^N$ and queue backlog $\{Q_i(t)\}_{i=1}^N$ are input into the DNN in each time frame. The DNN generates an output that indicates the proportion of partial offloading for WDs $\hat{\delta}^t$. To ensure exploitation and exploration, we quantize $\hat{\delta}^t$ into $(2K+1)$ candidate offloading decisions. We consider two types of quantization methods. One is fixed quantization interval, that is, take the $\hat{\delta}^t$ as the center and ϑ as the quantization interval to express as follows:

$$\{\hat{\delta}^t - K\vartheta, \ldots, \hat{\delta}^t - \vartheta, \hat{\delta}, \hat{\delta}^t + \vartheta, \ldots, \hat{\delta}^t + K\vartheta\}. \tag{17}$$

The other method is inspired by the noisy-order-preserving quantization approach [20], [23]. In contrast to the binary-offloading based noisy-order-preserving quantization method in the preceding literature, we propose partial-offloading based noisy-order-preserving quantization method that significantly diminishes the average data queue length. Utilizing binary offloading method, it takes a certain amount of time for the DNN to learn the optimal offloading strategy, during which the data queue length initially increases. As DNN approaches the optimal strategy, the data queue length decreases rapidly, ultimately stabilizing. The partial offloading approach facilitates a more expeditious and efficient learning process for the DNN to identify the optimal offloading strategy. The results of partial offloading decisions versus binary offloading decisions are shown in Fig. 4.

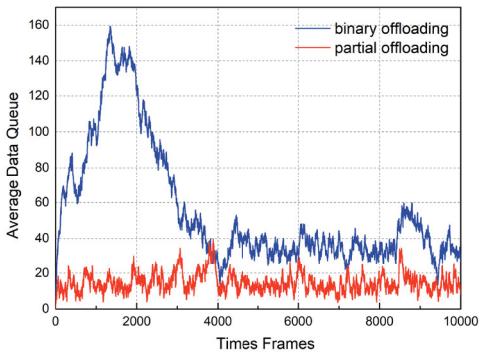


Fig. 4. The influence of different offloading decisions on data queue backlog.

The noisy-order-preserving quantization approach based on partial offloading is described as follows. The output action of the DNN network is $\hat{\delta}^t$. For the solution of the remaining K group of partial offloading decisions, calculate the distance of each element in $\hat{\delta}^t$ from 0.5 and arrange the distance values in ascending order to obtain an ordered arrangement of each element in $\hat{\delta}^t$:

$$|\delta_1 - 0.5| \leq |\delta_2 - 0.5| \leq \cdots \leq |\delta_N - 0.5| \tag{18}$$

where $a_1, a_2, \ldots, a_N$ is the permutation of the distance value relationship that conforms to formula (17). For the mth partial offloading decision $\delta_m^t = (\delta_{1m}, \delta_{2m}, \ldots, \delta_{Nm})$, the calculation method of δ_{im} is:

$$\delta_{im} = \begin{cases} \delta_{im} - a_m & 0 < \delta_{im} - a_m < 1 \\ -(\delta_{im} - a_m) & \delta_{im} - a_m < 0 \end{cases} \tag{19}$$

Then, Gaussian white noise is introduced to quantify the partial offloading decision, and $b_1, b_2, \ldots, b_N$ is the permutation of the distance value relationship conforming to

TABLE II
THE INFLUENCE OF DIFFERENT QUANTIFICATION METHODS ON THE EXPERIMENTAL RESULTS

	order-preserving quantization approach	noisy-order-preserving quantization approach
weighted sum computation rate	7.6573	7.9523
average data queue length	2.0842	3.4037

Formula (17). For the nth partial offloading decision $\delta_n^t = (\delta_{1n}, \delta_{2n}, \ldots, \delta_{Nn})$, the calculation method of δ_{in} is:

$$\delta_{in} = \begin{cases} \text{mod}(\delta_{in} - b_n) & \delta_{in} - b_n > 1 \\ \delta_{in} - b_n & 0 < \delta_{in} - b_n < 1 \\ |\text{mod}(\delta_{in} - b_n)| & \delta_{in} - b_n < 0 \end{cases} \tag{20}$$

The Appendix section provides schematic diagrams of the noise-preserving order quantization method, further explaining the implementation principle and key steps of this method through visual presentation.

We compare the experiments of the order preserving quantization methods with and without noise, and the experimental results are shown as follows. The experimental results are shown in the Table II. The experimental results show that, compared with the noise-free order-preserving quantization method, the noise-adding order-preserving quantization method can achieve a higher weighted sum calculation rate, and its queue backlog is within an acceptable range.

We conduct a series of experiments and finally verify that the latter quantitative method converges faster and performs better. Therefore, in the subsequent experiments, we adopt the noisy-order-preserving quantization method.

C. Model-Based Optimization Method for Resource Allocation

Once δ_i is given, the problem (2) is observed to become convex problem. We propose the Lagrangian duality method to address (a, τ), demonstrating a significantly reduced computational complexity in comparison to conventional convex optimization approaches, such as the interior point method. Firstly, Lagrange multiplier is introduced to form a partial Lagrange.

$$G(a, \tau_i, \epsilon) = \sum_{i=1}^{N}(Q_i(t) + V\omega_i)r_i^t + \epsilon(1 - a - \sum_{i=1}^{N}\tau_i), \tag{21}$$

where ϵ is the dual variable.

2) Critic model: We employ model-based optimization algorithms that formulate and solve computation offloading problems. Compared with traditional critic model using DNN, offloading actions can be evaluated more accurately. Specifically, the best action δ^t is obtained as

$$\delta^t = \arg\ \text{max} G(\delta_i^t, \xi^t), \tag{22}$$

where specific resource allocation method $G(\delta_i^t, \xi^t)$ is introduced in the next part. Algorithm 1 presents the proposed LyDROP algorithm based on Lyapunov optimization and DRL.

The corresponding dual function is

$$\begin{aligned} d(\epsilon) = \text{maximize}\ & G(a, \tau_i, \epsilon) && \text{(23a)} \\ \text{subject to}\ & D_i^t \le Q_i(t), && \text{(23b)} \\ & 0 \le a \le 1, 0 \le \tau_i \le 1, 0 \le \delta_i \le 1. && \text{(23c)} \end{aligned}$$

and the dual problem is

$$\underset{\epsilon}{\text{minimize}}\{d(\epsilon) | \epsilon \ge 0\}. \tag{24}$$

The dual problem realizes the same optimal objective value through the strong duality [14]. The optimal $\{a^*, \tau^*, \epsilon^*\}$ satisfies the formula in *Lemma 2*.

Proof: As stated in *Lemma 2*, the corresponding proof is furnished in the Appendix. ■

Lemma 2: The optimal $\{a^*, \tau^*, \epsilon^*\}$ follows the following equation:

$$\frac{\tau_i^*}{a^*} = \frac{\delta_i \lambda_0 (h_i^t)^2}{-(W(-\frac{1}{\exp(1+\frac{\epsilon}{\beta\lambda_2})}))^{-1} - 1}. \tag{25}$$

Algorithm 1 The LyDROP Algorithm

input: Parameter $V, \{\omega_i\}_{i=1}^N, n, N,$ training interval γ
output: joint action$\{\delta^t, \tau^*, a^*\}$

```
Initialize DNN with parameters;
Initialize data queue Q_i(1) = 0;
for t = 1, 2, ..., n do
    Input ξ^t = {h_i^t, Q_i(t)}_{i=1}^N;
    DNN output offloading action δ̂^t;
    Quantify partial offloading actions;
    Compute G(δ_i^t, ξ^t) in (P2) for every δ_i^t;
    Choose the optimal solution y^t = arg max G(δ_i^t, ξ^t);
    Execute the joint action {δ^t, τ^*, a^*};
    Renew replay memory with the addition of G(δ^t, ξ^t);
    if mod(t, γ) = 0 then
        Train the DNN with sample batch;
    end
t ← t + 1;
Renew queue backlog {Q_i(t)}_{i=1}^N;
end
```

Define $\Psi_i(\epsilon) \triangleq [-(W(-\frac{1}{\exp(1+\frac{\epsilon}{\beta\lambda_2})}))^{-1} - 1]^{-1}$, and $\Psi_i(\epsilon)$ with respect to ϵ is a monotonically decreasing function. Therefore,

$$\tau_i^* = \delta_i \lambda_0 (h_i^t)^2 a^* \Psi_i(\epsilon). \tag{26}$$

By combining with the previous formula, $\sum_{i=1}^{N} \tau_i^* + a^* = 1$, we get the formula for a^* with respect to ϵ,

$$a^* = \frac{1}{1 + \sum_{i=1}^{N} \delta_i \lambda_0 (h_i^t)^2 \Psi_i(\epsilon)} \triangleq \Gamma(\epsilon), \tag{27}$$

where $\Gamma(\epsilon)$ is a monotonically increasing function of ϵ. We then have the following *Lemma 3* on the optimal ϵ^*.

Proof: As stated in *Lemma 3*, the corresponding proof is furnished in the Appendix. ■

Lemma 3: There exists a best value ϵ^* that meets:

$$\Omega(\epsilon^*) \triangleq \frac{1}{3}((\Gamma(\epsilon^*)))^{-\frac{2}{3}} \sum_{i=1}^{N} \beta\lambda_1(1-\delta_i)^{\frac{1}{3}}\left(\frac{h_i^t}{k_i}\right)^{\frac{1}{3}} + \sum_{i=1}^{N} \frac{\beta\lambda_2\delta_i\lambda_0(h_i^t)^2}{1+1/\Lambda(\epsilon^*)} - \epsilon^*. \tag{28}$$

$\Omega(\epsilon)$ decreases monotonically with respect to ϵ. The optimal ϵ^* can be efficiently obtained by using dichotomy. After identifying the optimal ϵ^*, the optimal $\{\tau^*, a^*\}$ can be easily calculated. The pseudo-code representing the aforementioned method is presented in Algorithm 2.

Algorithm 2 Model-Based Optimization Method for Resource Allocation

input : partial offloading actions
output : the optimal$\{\tau^*, a^*\}$

initialize: $\iota \leftarrow 1, UB \leftarrow 10^4, LB \leftarrow 0$;
repeat
  $\epsilon \leftarrow \frac{UB+LB}{2}$;
  if $\Omega(\epsilon) > 0$ in (28) **then**
    $LB \leftarrow \epsilon$;
  else
    $UB \leftarrow \epsilon$;
  end
until $|UB - LB| \leq \iota$;
Calculate a^* using (27) and τ^* using (26);
Return $\{\tau^*, a^*\}$.

D. Complexity Analysis of LyDROP Algorithm

The computational complexity of the LyDROP algorithm is predominantly attributed to two critical components: the quantification of offloading decisions and the optimization of resource allocation strategies. We quantize $\hat{\delta}^t$ into $(2K+1)$ candidate offloading decisions. The computational complexity of dichotomy in resource allocation strategy is $O(N\log_2(\frac{UB}{\iota}))$. The complexity of LyDROP algorithm in each time frame is $O(nN\log_2(\frac{UB}{\iota})(2K+1))$, that is, $O(nNK\log_2(\frac{UB}{\iota}))$. The LyDROP algorithm exhibits remarkably low computational times, thereby demonstrating exceptional efficiency in practical scenarios. Subsequent chapter will present a comparative analysis of the algorithm's performance against serval baseline or state-of-the-art algorithms.

V. Simulation Experiments

We conduct a series of simulation experiments and furnish simulation results in the present section based on Tensorflow 2.13 and Python 3.9, aiming at evaluating the performance of the LyDROP algorithm.

Unless a different statement is clearly provided, Table III introduces the fundamental experiment parameters. Supposed that h_i^t follows an i.i.d. Rician distribution and line-of-sight link gain is defined as 0.3 times average channel gain, denoted by $\bar{h_i}$, where $\bar{h_i} = A_d(\frac{3\times10^8}{4\pi f_c d_i})^{d_e}, i = 1,\ldots,N$ [23]. In the above formula, the antenna gain A_d is set to 3, the carrier frequency f_c is set to 915 MHz and the path loss exponent d_e is set to 3. Meanwhile, we assume in the model that the positions of the WDs and AP are all fixed, d_i in meters is utilized to represent the distance between the ith WD and the AP. The task arrivals for all WDs follow exponential distribution and average rate is equal, i.e., $\mathbb{E}[R_i^t] = \kappa_i, i = 1,\ldots,N$.

TABLE III
Simulation Parameter Setting

Description	Parameter	Value
The wireless channel's bandwidth	B	2 MHz
The energy efficiency coefficient	k_i	10^{-26}
The energy harvesting efficiency	σ	0.51
The AP's transmitted power	P_a	3 W
The number of cycles needed to process one bit of task data	φ	100
The receiver noise power	N_0	10^{-10} W
The communication overhead	χ	1.1
The length of a time frame	T	1 S
The penalty factor	V	36

TABLE IV
CPU Computation Time With Different K Values

K value	CPU computation time
K=8	0.1833
K=9	0.16366
K=10	0.1551

We introduce the design method of adaptive K value proposed in [14] and compare the influence of different K values on CPU computation time. The experimental results are shown in the Table IV. And we conduct experimental verification by taking different values of K. The experimental results are shown in Fig. 5. By combining Table IV and Fig. 5, it can be seen that when$K = 10$, the weighted sum computation rate is the largest, the average data queue length is the smallest, and the CPU computation time is the smallest. Therefore, we take $K = 10$ as the number of noise-preserving order quantization groups.

Similarly, we conduct experimental verification by taking different values of V. The experimental results are shown in Fig. 6. During the experiment, the average task arrival rate is set at 8 Mbps, that is, $\kappa_i = 8$ Mbps. The weighted sum calculation rate increases with the increase of the V value, which is consistent with our objective function. We select an appropriate V value for the experiment to make the weighted sum computation rate as large as possible and keep the average data queue length within an acceptable range.

We compare the new LyDROP with three benchmarks, namely, "LyCD", "LyRPO" and "Myopic".

1) LyCD: Firstly, Lyapunov optimization is used to transform problem (P1) into problem (P2). When solving the deterministic problem (P2) for each time frame, a set of offloading decisions $\{\delta_1, \delta_2, \ldots, \delta_N\}$ are randomly generated first. Then, the coordinate descent method is used to update the offloading decisions, and the Lagrange duality method is utilized to solve the WPT time and transmission time corresponding to each group of offloading decisions, in order to find the optimal resource allocation scheme. The idea of coordinate descent is that a multivariable function can be optimized in one direction at a time to obtain an extreme value.

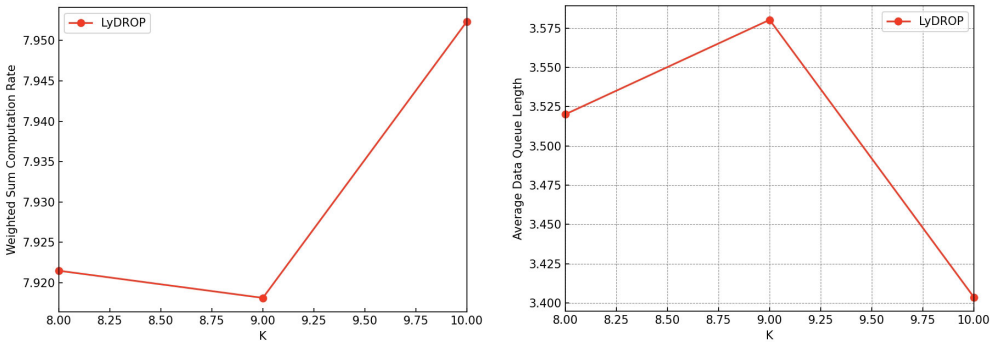


Fig. 5. Experimental results of the LyDROP algorithm with different K values.

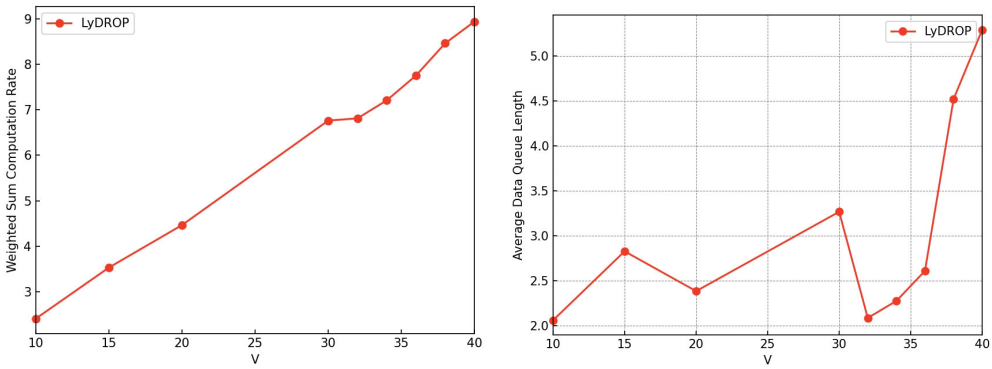


Fig. 6. Experimental results of the LyDROP algorithm with different V values.

2) LyRPO: Similar to the above algorithm, the problem (P1) is transformed into problem (P2) using Lyapunov optimization, and then the optimization objectives are realized by randomly generating partial offloading strategy for all WDs.

3) Myopic: The Myopic method disregards queue backlog, with the aim of maximizing the weighted sum computation rate by resolving the following:

$$\underset{\delta,\tau,a}{\text{maximize}} \quad \sum_{i=1}^{N} \omega_i r_i^t \tag{29a}$$

$$\text{subject to} \quad (16b) - (16d). \tag{29b}$$

Fig. 7(a) depicts the weighted sum computation rate and Fig. 7(b) depicts queue backlog of online WP-MEC system under different algorithms when the number of WDs and time are 10, 10000, respectively. Set $\kappa_i = 8$ Mbps. Set $\omega_i = 1$, that is, the priority of wireless devices is not taken into account. It is observed that in Fig. 7(a) LyDROP algorithm and Myopic algorithm have better weighted sum computation rate than the other two algorithms. Myopic algorithm does not consider queue backlog, so in Fig. 7(b) it can not control data queue stability, nor can it reduce the queue backlog. Therefore, when increasing the number of WDs, Myopic algorithm is not taken into account.

Fig. 8 illustrates the performance of LyDROP algorithm along with the comparison algorithms when $N = 20$. Obviously, our algorithm demonstrates superiority over LyCD algorithm and LyRPO algorithm. Fig. 9 presents the performance of LyDROP algorithm along with the comparison algorithms when $N = 30$. Combined with Fig. 7 and Fig. 8, the superiority of LyDROP algorithm over alternative approaches becomes increasingly pronounced as the number of WDs increases. Additionally, LyCD algorithm demonstrates superior performance relative to the other two algorithms. For the purpose of further verifying the performance of our algorithm, we conduct a comparative analysis of the CPU calculation time of LyDROP and LyCD algorithms with different number of WDs. The CPU calculation time is defined as the quotient of the program's total execution duration and the set total time.

TABLE V
CPU COMPUTATION TIME VERSUS THE NUMBER OF WDS

	LyDROP	LyCD	$\frac{\text{LyCD}}{\text{LyDROP}}$
N=10	0.18	0.12	0.67
N=20	0.44	1.62	3.68
N=30	0.56	9.18	16.39

As shown in Table V, the CPU calculation time difference between LyDROP algorithm and LyCD algorithm is small when the number of WDs is small. The CPU calculation time gap between the two algorithms gradually increases with the increase of the number of WDs. When $N = 20$, the LyCD algorithm takes 3.68 times longer than the LyDROP algorithm. When $N = 30$, the value increases to 16.39 times. To sum up, the proposed LyDROP algorithm demonstrates superior applicability to online WP-MEC system with random channels and time-varying task arrivals, particularly in scenario with a large number of WDs.

Fig. 10 and Fig. 11 consider the performance of different algorithms as the quantity of WDs increases in the case that

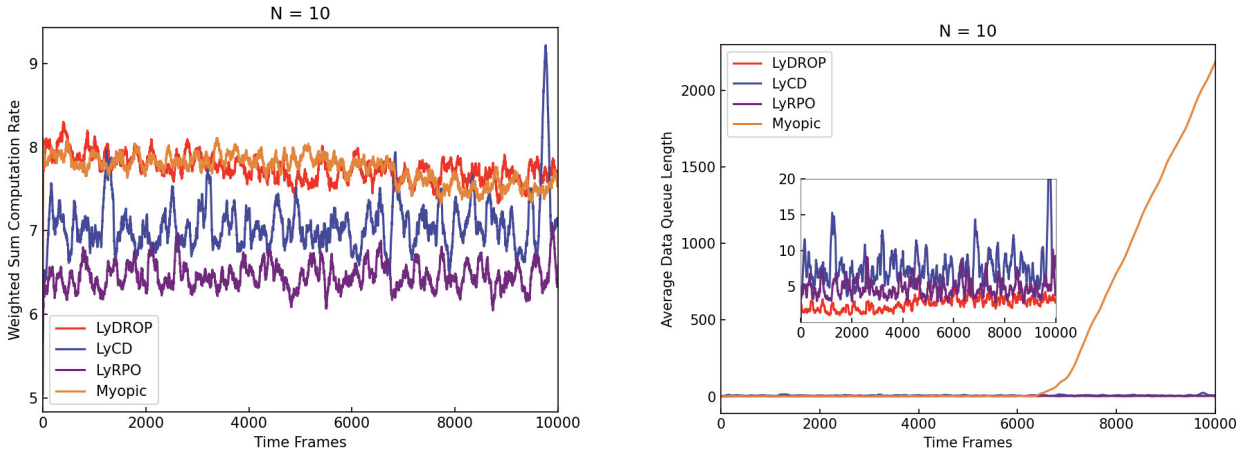


Fig. 7. The experimental results of each algorithm when N=10, n=10000. (a) The weighted sum computation rate. (b) Queue backlog.

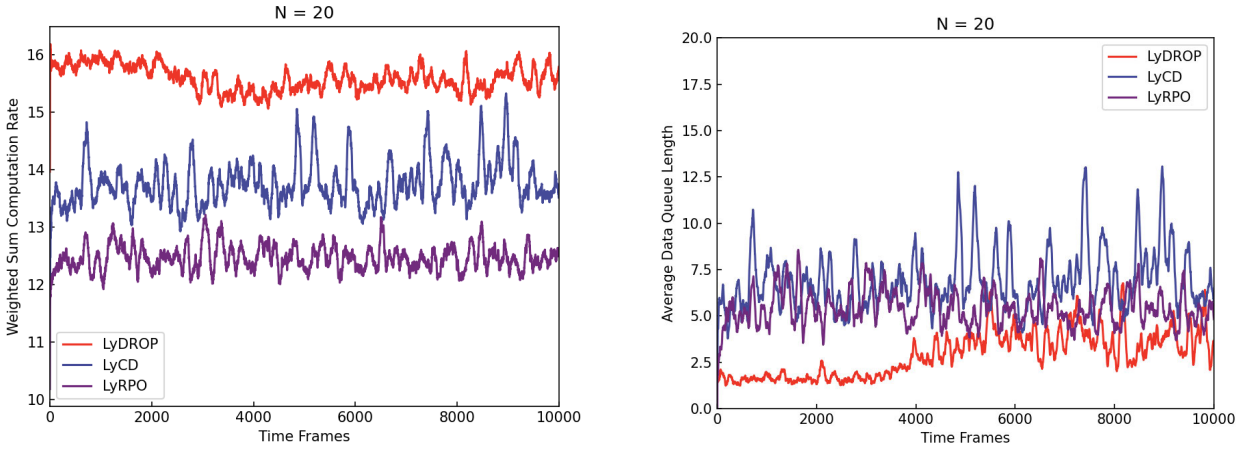


Fig. 8. The experimental results of each algorithm when N=20, n=10000. (a) The weighted sum computation rate. (b) Queue backlog.

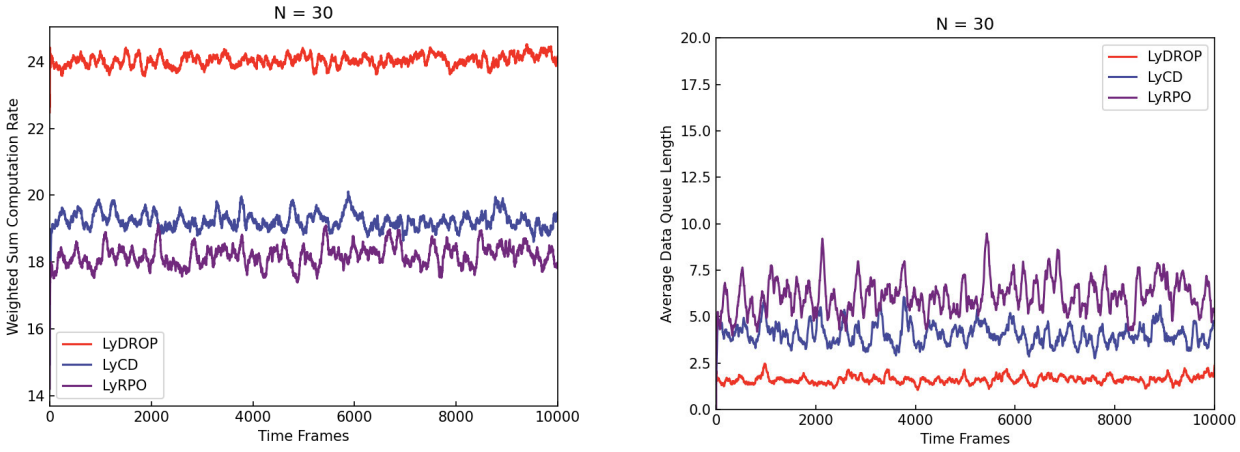


Fig. 9. The experimental results of each algorithm when N=30, n=10000. (a) The weighted sum computation rate. (b) Queue backlog.

some WDs devices have priority. Considering the system is applied to IIoTs, such as forest fire detection [39]. For the purpose of ensuring the rapid response of forest fire monitoring, the smoke sensor has a higher priority relative to humidity sensor. Specifically, the WDs are numbered successively, then ω_i is set to 1.2 for even-numbered WDs and ω_i of the odd-numbered WDs is set to 1. Fig. 10 depicts the weighted sum computation rate of the four algorithms for different number of WDs. It is obvious that compared with the other three algorithms, the weighted sum computation rate of LyDROP algorithm has significant advantages. Fig. 11 depicts the queue backlog of the four algorithms for different number of WDs. It is found that the queue backlog of Myopic algorithm fluctuates greatly, and the stability of the data queue can not be guaranteed. Compared with the other two algorithms, our algorithm guarantees a smaller queue backlog.

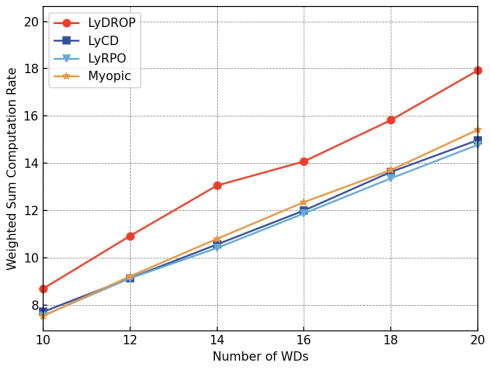


Fig. 10. The weighted sum computation rate versus the number of WDs.

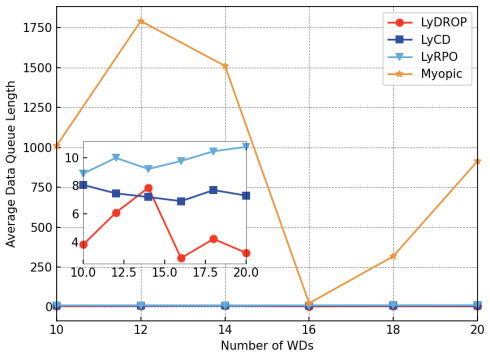


Fig. 11. The queue backlog versus the number of WDs.

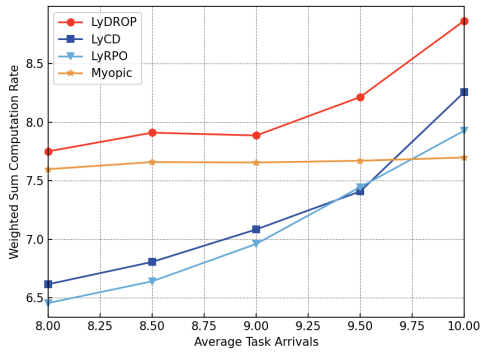


Fig. 12. The weighted sum computation rate versus the average task arrivals.

Fig. 12 and Fig. 13 depict the comparison results of the four algorithms under different task arrival conditions. Fig. 12 depicts the relationship between weighted sum computation rate and average task arrivals. As the number of tasks arriving at each WD increases, Myopic algorithm's weighted sum computation rate does not change significantly. The other three algorithms' weighted sum computation rate increases with the increasing number of tasks reaching each WD, and our algorithm performs better than the remaining two algorithms.

Fig. 13 depicts the relationship between the queue backlog and the average task arrivals for the four algorithms. The Myopic algorithm's queue backlog is at least two orders of magnitude larger compared to other algorithms when the average number of tasks arrived increases from 8 Mbps to 8.25 Mbps. This difference changes dramatically as the number of tasks increases, so subsequent unstable points are not plotted. In the process of increasing the average task arrivals

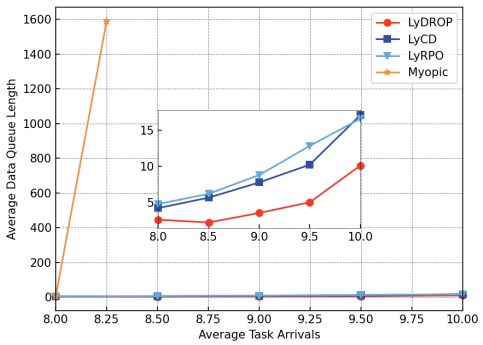


Fig. 13. The queue backlog versus the average task arrivals.

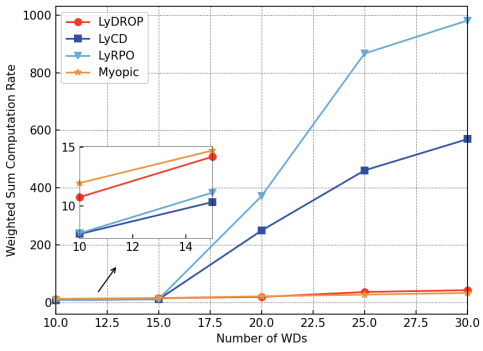


Fig. 14. The weighted sum computation rate versus the number of WDs.

from 8 Mbps to 10 Mbps, the queue backlog of LyDROP algorithm is smaller than that of LyCD algorithm and LyRPO algorithm. Fig. 12 and Fig. 13 show that our algorithm is also suitable for scenarios with a large amount of tasks randomly arriving at WDs, and the performance of LyDROP algorithm is superior to other three comparison algorithms in all aspects.

We utilize a real-world dataset of Melbourne in Australia.[1] We randomly select a BS with latitude and longitude coordinates at (−37.81517, 144.97476). From the 816 user groups that are located within the latitude range of [−37.8209, −37.8078] and the longitude range of [144.9517, 144.9744], we select different number of users who are close to the above BS for experimental verification. Fig. 14 and Fig. 15 show the comparison results of the four algorithms with different user quantities. In the previous simulation experiments, it is assumed that the distances between the users and the AP are equally spaced [23]. However, the actual location distribution of real users is not regular.

Based on the real user location information, all the four algorithms maintain good performance in the case of a small number of users (such as 10 users). The proposed LyDROP algorithm maintains relatively higher weighted sum computation rate and lower queue backlog. With an increasing number of users, the overall performance of LyCD algorithm and LyRPO algorithm deteriorates, as shown in Fig. 14 and Fig. 15 Due to the randomness of the offloading strategy of LyRPO algorithm, it can not provide a good offloading strategy when faced with a multitude of users. Consequently, LyRPO algorithm's queue backlog is significantly higher than LyCD algorithm's queue backlog and other two algorithms'

[1]https://github.com/swinedge/eua-dataset

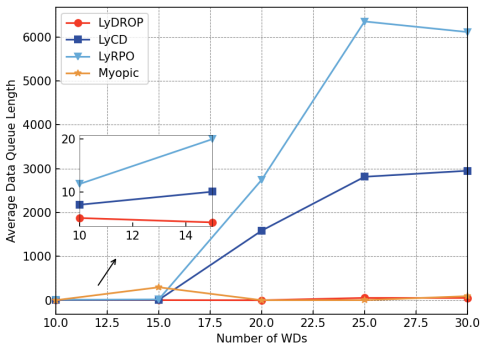


Fig. 15. The queue backlog versus the number of WDs.

queue backlog. A large queue backlog means that there are users whose data is not being processed in time. Different from the LyRPO algorithm, the Myopic algorithm obtains offloading decision through interaction between the DNN and environment. The Myopic algorithm does not consider queue backlog, so the algorithm performance is unstable. When the user count is 15, the queue backlog of Myopic algorithm reaches 296, which is much higher than the queue backlog of the other three algorithms under the same user conditions. The proposed LyDROP algorithm exhibits superior performance compared to the other three algorithms. With an increasing number of users, the performance of the LyDROP algorithm is stable, that is, it maintains a high weighted sum computation rate and a low data queue backlog. In addition, the LyDROP algorithm takes less time than the LyCD algorithm, and the more users there are, the more obvious the time gap is. This means that LyDROP algorithm formulates the optimal offloading strategy more effectively and allocates resources more quickly in scenarios with a large number of users.

VI. Conclusion

We conduct an in-depth exploration of the online computation offloading problem for WP-MEC system in this paper. For the purpose of maximizing the weighted sum computation rate of all WDs while reducing the average data queue length as much as possible, we introduce an innovative online algorithm named LyDROP. The LyDROP algorithm jointly optimizes the WPT duration time, partial offloading decisions as well as transmission time allocation. Simulation outcomes demonstrate that LyDROP algorithm significantly surpasses existing baseline approaches concerning both weighted sum computation rate and queue backlog. As previously considered, the research in this work is expected to expand applications such as the IIoTs and UAVs. At present, the proposed WP-MEC system considers one AP and WDs in fixed locations, and a possible extension is to consider scenarios where multiple APs exist and wireless devices are mobile. One of the key issues in multi-AP multi-user scenarios is whether multiple APs supply power to WDs at the same time or only one AP at a time and how to select the AP appropriately. Consequently, the LyDROP algorithm needs further optimization to effectively address the aforementioned challenges. We will attempt to investigate online computation offloading problems for more complex WP-MEC scenarios of multiple edge servers in future research work. In addition, it is possible to consider introducing the recently highly regarded AI large model technology to solve complex constraint problems. It is also possible to consider designing lightweight AI algorithms to adapt to the computing and storage resource limitations of edge devices, ensuring the feasibility and efficiency of the algorithms in actual deployment.

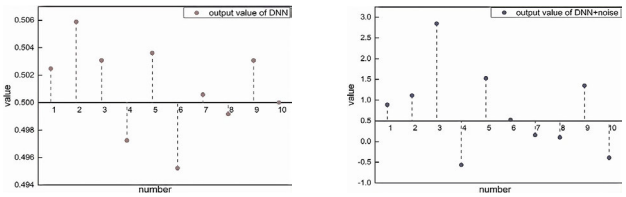


Fig. 16. Schematic diagram of the noisy-order-preserving quantization approach.

Appendix
Proof of Lemma 1

Based on the triangle inequality, we have

$$Q_i(t+1)^2 \le Q_i(t)^2 + (R_i^t(t))^2 + (D_i^t(t))^2 + 2Q_i(t)(R_i^t(t) - D_i^t(t)). \tag{30}$$

Summing the aforementioned inequality over all $i(i = 1, \ldots, N)$, the expression is

$$\frac{1}{2}\sum_{i=1}^{N}(Q_i(t+1)^2 - Q_i(t)^2) \le \frac{1}{2}\sum_{i=1}^{N} D_i(t)^2 + \frac{1}{2}\sum_{i=1}^{N} R_i(t)^2 + Q_i(t)(R_i^t(t) - D_i^t(t)). \tag{31}$$

Substituting (30) and (31) into the conditional Lyapunov drift function (13), we derive

$$\triangle L(\Theta(t)) \le \quad C - \frac{1}{2}\sum_{i=1}^{N}\mathbb{E}\{Q_i(t)(R_i^t - D_i^t)|\Theta(t)\}, \tag{32}$$

where $C = \sum_{i=1}^{N}((R_{i,max}^t)^2 + (D_{i,max}^t)^2)$, $R_{i,max}^t$ and $D_{i,max}^t$ are the upper bound of R_i^t and D_i^t, respectively. Thus,

$$\triangle_V L(\Theta(t)) \le C + \mathbb{E}\{Q_i(t)(R_i^t - D_i^t)|\Theta(t)\} - \mathbf{V}\mathbb{E}\{\omega_i r_i^t|\Theta(t)\}. \tag{33}$$

Appendix
Noisy-order-preserving Quantization

TThe essence of the partial offloading noise order-preserving quantization method is non-uniform quantization, that is, the quantization interval is not fixed.

As shown in Fig. 16, the first graph shows a set of partial offloading decisions output by the neural network. The tenth number in the first graph is close to 0.5, and then we obtain this distance value, and subtract this value from this set of numbers to get a new set of offloading decisions. The second figure shows a set of partial offloading decisions mixed with noise. The sixth number in the second graph is close to 0.5, again we take its distance value and subtract it from the set of numbers. Then the partial offloading decision with noise can obtain by taking the residual operation and the absolute value operation.

APPENDIX
PROOF OF LEMMA 2

Find the first partial derivative of G with respect to τ_i as follows:

$$\frac{\partial G}{\partial \tau_i} = (Q_i(t) + V\omega_i)[\frac{B}{\chi \ln 2} \ln (1 + \frac{\delta_i \sigma P_a (h_i^t)^2 a}{\tau_i N_0}) + \frac{-B\tau_i^{-1}}{\chi \ln 2} \frac{\delta_i \sigma P_a (h_i^t)^2 a}{N_0} \frac{1}{1 + \frac{\delta_i \sigma P_a (h_i^t)^2 a}{\tau_i N_0}}] - \epsilon. \quad (34)$$

If $\frac{\partial G}{\partial \tau_i} = 0$, it is at the maximum value. Let $\beta = Q_i(t) + V\omega_i$, $\lambda_0 = \frac{\sigma P_a}{N_0}$, $\lambda_1 = \frac{(\sigma P_a)^{\frac{1}{3}}}{\varphi}$, $\lambda_2 = \frac{B}{\chi \ln 2}$, then

$$\ln (1 + \lambda_0 \delta_i (h_i^t)^2 a \tau_i^{-1}) = (1 + \frac{\epsilon}{\beta \lambda_2}) - \frac{1}{1 + \delta_i \lambda_0 (h_i^t)^2 a \tau_i^{-1}}. \quad (35)$$

Take the logarithm base e of both sides of the above equation,

$$(1 + \lambda_0 \delta_i (h_i^t)^2 a \tau_i^{-1}) \exp (\frac{1}{1 + \delta_i \lambda_0 (h_i^t)^2 a \tau_i^{-1}}) = \exp (1 + \frac{\epsilon}{\beta \lambda_2}). \quad (36)$$

According to the Lambert-W function, if $-x \exp -x = -\frac{1}{z}$, then $x = -W(-\frac{1}{z})$. So we get the following formula,

$$\frac{1}{1 + \delta_i \lambda_0 (h_i^t)^2 a \tau_i^{-1}} = -W(-\frac{1}{\exp (1 + \frac{\epsilon}{\beta \lambda_2})}). \quad (37)$$

APPENDIX
PROOF OF LEMMA 3

The first partial derivative of G with respect to a is

$$\frac{\partial G}{\partial a} = \sum_{i=1}^{N} \frac{1}{3} \beta \lambda_1 (1 - \delta_i)^{\frac{1}{3}} (\frac{h_i^t}{k_i})^{\frac{1}{3}} a^{(-\frac{2}{3})} + \sum_{i=1}^{N} \frac{\beta \lambda_2 \delta_i \lambda_0 (h_i^t)^2}{1 + \delta_i \lambda_0 (h_i^t)^2 a \tau_i^{-1}} - \epsilon. \quad (38)$$

The goal is to set the first partial derivative to be zero to find the extreme point. Define $\delta_i \lambda_0 (h_i^t)^2 a \tau_i^{-1} = \frac{1}{\Lambda(\epsilon^*)}$. Hence,

$$\Omega(\epsilon^*) \triangleq \frac{1}{3} ((\Gamma(\epsilon^*)))^{-\frac{2}{3}} \sum_{i=1}^{N} \beta \lambda_1 (1 - \delta_i)^{\frac{1}{3}} (\frac{h_i^t}{k_i})^{\frac{1}{3}} + \sum_{i=1}^{N} \frac{\beta \lambda_2 \delta_i \lambda_0 (h_i^t)^2}{1 + 1/\Lambda(\epsilon^*)} - \epsilon^*. \quad (39)$$

REFERENCES

[1] L. Shi, Y. Ye, X. Chu, and G. Lu, "Computation energy efficiency maximization for a NOMA-based WPT-MEC network," *IEEE Internet Things J.*, vol. 8, no. 13, pp. 10731–10744, Jul. 2021.

[2] M. Zeng, R. Du, V. Fodor, and C. Fischione, "Computation rate maximization for wireless powered mobile edge computing with NOMA," in *Proc. IEEE 20th Int. Symp. World Wireless, Mobile Multimedia Netw.*, Jun. 2019, pp. 1–9.

[3] T. Nguyen Dang, A. Manzoor, Y. K. Tun, S. M. A. Kazmi, Z. Han, and C. S. Hong, "A contract-theory-based incentive mechanism for UAV-enabled VR-based services in 5G and beyond," *IEEE Internet Things J.*, vol. 10, no. 18, pp. 16465–16479, Sep. 2023.

[4] F. Liu, J. Chen, Q. Zhang, and B. Li, "Online MEC offloading for V2V networks," *IEEE Trans. Mobile Comput.*, vol. 22, no. 10, pp. 6097–6109, Oct. 2023.

[5] S. Mao, S. Leng, S. Maharjan, and Y. Zhang, "Energy efficiency and delay tradeoff for wireless powered mobile-edge computing systems with multi-access schemes," *IEEE Trans. Wireless Commun.*, vol. 19, no. 3, pp. 1855–1867, Mar. 2020.

[6] A. Alabsi et al., "Wireless power transfer technologies, applications, and future trends: A review," *IEEE Trans. Sustain. Comput.*, vol. 10, no. 1, pp. 1–18, Jan. 2025.

[7] L. Sun, L. Wan, K. Liu, and X. Wang, "Cooperative-evolution-based WPT resource allocation for large-scale cognitive industrial IoT," *IEEE Trans. Ind. Informat.*, vol. 16, no. 8, pp. 5401–5411, Aug. 2020.

[8] X. Wang, Z. Ning, L. Guo, S. Guo, X. Gao, and G. Wang, "Online learning for distributed computation offloading in wireless powered mobile edge computing networks," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 8, pp. 1841–1855, Aug. 2022.

[9] H. Long, C. Xu, G. Zheng, and Y. Sheng, "Socially-aware energy-efficient task partial offloading in MEC networks with D2D collaboration," *IEEE Trans. Green Commun. Netw.*, vol. 6, no. 3, pp. 1889–1902, Sep. 2022.

[10] Y. He, X. Wu, Z. He, and M. Guizani, "Energy efficiency maximization of backscatter-assisted wireless-powered MEC with user cooperation," *IEEE Trans. Mobile Comput.*, vol. 23, no. 2, pp. 1878—1887, Feb. 2023.

[11] B. Li, F. Si, W. Zhao, and H. Zhang, "Wireless powered mobile edge computing with NOMA and user cooperation," *IEEE Trans. Veh. Technol.*, vol. 70, no. 2, pp. 1957–1961, Feb. 2021.

[12] S. Zeng, X. Huang, and D. Li, "Joint communication and computation cooperation in wireless-powered mobile-edge computing networks with NOMA," *IEEE Internet Things J.*, vol. 10, no. 11, pp. 9849–9862, Jun. 2023.

[13] L. Zhang, Q. Song, M. Wu, W. Qi, and L. Guo, "Joint terminal pairing and multi-dimensional resource allocation for cooperative computation in a WP-MEC system," *IEEE Trans. Green Commun. Netw.*, vol. 7, no. 3, pp. 1447–1456, Sep. 2023.

[14] S. Bi and Y. J. Zhang, "Computation rate maximization for wireless powered mobile-edge computing with binary computation offloading," *IEEE Trans. Wireless Commun.*, vol. 17, no. 6, pp. 4177–4190, Jun. 2018.

[15] L. Sun, L. Wan, J. Wang, L. Lin, and M. Gen, "Joint resource scheduling for UAV-enabled mobile edge computing system in Internet of Vehicles," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 12, pp. 15624–15632, Dec. 2023.

[16] X. Li, L. Huang, H. Wang, S. Bi, and Y. A. Zhang, "An integrated optimization-learning framework for online combinatorial computation offloading in MEC networks," *IEEE Wireless Commun.*, vol. 29, no. 1, pp. 170–177, Feb. 2022.

[17] Z. Hu et al., "An efficient online computation offloading approach for large-scale mobile edge computing via deep reinforcement learning," *IEEE Trans. Services Comput.*, vol. 15, no. 2, pp. 669–683, Mar. 2022.

[18] W. Shang et al., "The impact of deep reinforcement learning-based traffic signal control on emission reduction in urban road networks empowered by cooperative vehicle-infrastructure systems," *Appl. Energy*, vol. 390, Jul. 2025, Art. no. 125884.

[19] Q. Dong, J. Tang, W.-L. Shang, X. Shao, and M. Chatzimichailidou, "Intelligent scheduling of urban rail transit loop line trains: A study on coordinated optimization of timetables and rolling stock circulation plans based on DQN deep reinforcement learning," *Comput. Ind. Eng.*, vol. 207, Sep. 2025, Art. no. 111297.

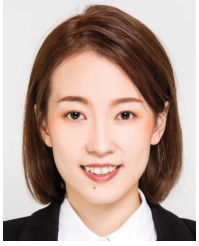
[20] L. Huang, S. Bi, and Y. A. Zhang, "Deep reinforcement learning for online computation offloading in wireless powered mobile-edge computing networks," *IEEE Trans. Mobile Comput.*, vol. 19, no. 11, pp. 2581–2593, Nov. 2020.

[21] K. Zheng, G. Jiang, X. Liu, K. Chi, X. Yao, and J. Liu, "DRL-based offloading for computation delay minimization in wireless-powered multi-access edge computing," *IEEE Trans. Commun.*, vol. 71, no. 3, pp. 1755–1770, Mar. 2023.

[22] S. Zhang, H. Gu, K. Chi, L. Huang, K. Yu, and S. Mumtaz, "DRL-based partial offloading for maximizing sum computation rate of wireless powered mobile edge computing network," *IEEE Trans. Wireless Commun.*, vol. 21, no. 12, pp. 10934–10948, Dec. 2022.

[23] S. Bi, L. Huang, H. Wang, and Y. A. Zhang, "Lyapunov-guided deep reinforcement learning for stable online computation offloading in mobile-edge computing networks," *IEEE Trans. Wireless Commun.*, vol. 20, no. 11, pp. 7519–7537, Nov. 2021.

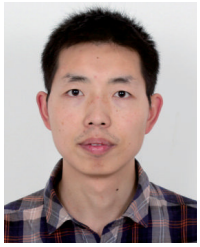
[24] X. Chen et al., "Augmented deep reinforcement learning for online energy minimization of wireless powered mobile edge computing," *IEEE Trans. Commun.*, vol. 71, no. 5, pp. 2698–2710, May 2023.
[25] H. Hu, X. Zhou, Q. Wang, and R. Q. Hu, "Online computation offloading and trajectory scheduling for UAV-enabled wireless powered mobile edge computing," *China Commun.*, vol. 19, no. 4, pp. 257–273, Apr. 2022.
[26] M. Bolourian and H. Shah-Mansouri, "Energy-efficient task offloading for three-tier wireless-powered mobile-edge computing," *IEEE Internet Things J.*, vol. 10, no. 12, pp. 10400–10412, Jun. 2023.
[27] X. Wang, J. Li, Z. Ning, Q. Song, L. Guo, and A. Jamalipour, "Wireless powered metaverse: Joint task scheduling and trajectory design for multi-devices and multi-UAVs," *IEEE J. Sel. Areas Commun.*, vol. 42, no. 3, pp. 552–569, Mar. 2024.
[28] M. Guo, W. Wang, X. Huang, Y. Chen, L. Zhang, and L. Chen, "Lyapunov-based partial computation offloading for multiple mobile devices enabled by harvested energy in MEC," *IEEE Internet Things J.*, vol. 9, no. 11, pp. 9025–9035, Jun. 2022.
[29] J. Zhang, J. Du, Y. Shen, and J. Wang, "Dynamic computation offloading with energy harvesting devices: A hybrid-decision-based deep reinforcement learning approach," *IEEE Internet Things J.*, vol. 7, no. 10, pp. 9303–9317, Oct. 2020.
[30] H. Wu, H. Tian, S. Fan, and J. Ren, "Data age aware scheduling for wireless powered mobile-edge computing in industrial Internet of Things," *IEEE Trans. Ind. Informat.*, vol. 17, no. 1, pp. 398–408, Jan. 2021.
[31] H. Wu, X. Lyu, and H. Tian, "Online optimization of wireless powered mobile-edge computing for heterogeneous industrial Internet of Things," *IEEE Internet Things J.*, vol. 6, no. 6, pp. 9880–9892, Dec. 2019.
[32] X. Deng, J. Li, L. Shi, Z. Wei, X. Zhou, and J. Yuan, "Wireless powered mobile edge computing: Dynamic resource allocation and throughput maximization," *IEEE Trans. Mobile Comput.*, vol. 21, no. 6, pp. 2271–2288, Jun. 2022.
[33] A. Javadpour, G. Wang, and S. Rezaei, "Resource management in a peer to peer cloud network for IoT," *Wireless Pers. Commun.*, vol. 115, no. 3, pp. 2471–2488, Dec. 2020.
[34] A. Javadpour, A. Nafei, F. Ja'fari, P. Pinto, W. Zhang, and A. K. Sangaiah, "An intelligent energy-efficient approach for managing IoE tasks in cloud platforms," *J. Ambient Intell. Humanized Comput.*, vol. 14, no. 4, pp. 3963–3979, Apr. 2023.
[35] A. Javadpour et al., "An energy-optimized embedded load balancing using DVFS computing in cloud data centers," *Comput. Commun.*, vol. 197, pp. 255–266, Jan. 2023.
[36] K. Zhu et al., "Aerial refueling: Scheduling wireless energy charging for UAV enabled data collection," *IEEE Trans. Green Commun. Netw.*, vol. 6, no. 3, pp. 1494–1510, Sep. 2022.
[37] S. Guo, B. Xiao, Y. Yang, and Y. Yang, "Energy-efficient dynamic offloading and resource scheduling in mobile cloud computing," in *Proc. IEEE 35th Annu. IEEE Int. Conf. Comput. Commun.*, Apr. 2016, pp. 1–9.
[38] M. Neely, *Stochastic Network Optimization with Application to Communication and Queueing Systems*. San Rafael, CA, USA: Morgan & Claypool, 2010.
[39] L. Sun, L. Wan, and X. Wang, "Learning-based resource allocation strategy for industrial IoT in UAV-enabled MEC systems," *IEEE Trans. Ind. Informat.*, vol. 17, no. 7, pp. 5031–5040, Jul. 2021.



Lu Sun (Member, IEEE) received the B.S. and Ph.D. degrees from the School of Software, Dalian University of Technology, Dalian, China, in 2013 and 2019, respectively. She is currently an Associate Professor at the Institute of Information Science Technology, Dalian Maritime University, Dalian. Her current research interests include intelligent computation, mobile edge computing, machine learning, and resource allocation.

Rina Liang received the bachelor's degree in communication engineering from Dalian Minzu University, Liaoning, China, in 2021. She is currently pursuing the master's degree with the School of Information Science and Technology, Dalian Maritime University, Liaoning. Her research interests include online computation offloading, deep reinforcement learning, WPT, and MEC.

Liangtian Wan (Member, IEEE) received the B.S. and Ph.D. degrees from the College of Information and Communication Engineering, Harbin Engineering University, Harbin, China, in 2011 and 2015, respectively. From October 2015 to April 2017, he was a Research Fellow at the School of Electrical and Electronic Engineering, Nanyang Technological University, Singapore. He is currently an Associate Professor at the School of Software, Dalian University of Technology, China. His current research interests include graph learning, data science, and array signal processing. He has been serving as an Associate Editor for IEEE ACCESS and *Journal of Information Processing Systems*.



Kaihui Liu (Member, IEEE) received the Ph.D. degree from the National Key Laboratory of Science and Technology on Communications, University of Electronic Science and Technology of China, in 2019. His research interests include the mathematics of data science, particularly the interplay of convex and nonconvex optimization, statistical learning and estimation, signal processing, computational imaging, and wireless communication.

Zhaolong Ning (Senior Member, IEEE) received the Ph.D. degree from Northeastern University, China, in 2014. He was a Research Fellow at Kyushu University, Japan, from 2013 to 2014. He is currently a Full Professor with the School of Communication and Information Engineering, Chongqing University of Posts and Telecommunications, Chongqing, China. He has published more than 150 scientific papers in international journals and conferences. His research interests include mobile edge computing, 6G networks, machine learning, and resource management. He serves as an Associate Editor or a Guest Editor for several journals, such as IEEE TRANSACTIONS ON INDUSTRIAL INFORMATICS, IEEE TRANSACTIONS ON COMPUTATIONAL SOCIAL SYSTEMS, and IEEE INTERNET OF THINGS JOURNAL. He has been a highly cited researcher (Web of Science) since 2020.

Jie Wang (Senior Member, IEEE) received the B.S. degree in electronic engineering from Dalian University of Technology, Dalian, China, in 2003, the M.S. degree in electronic engineering from Beihang University, Beijing, China, in 2006, and the Ph.D. degree in electronic engineering from Dalian University of Technology, Dalian, in 2011. He was a Visiting Researcher with the University of Florida, Gainesville, FL, USA, from 2013 to 2014. He is currently a Professor with the School of Information Science and Technology, Dalian Maritime University, Dalian. His research interests include wireless localization and tracking, radio tomography, wireless sensing, machine learning, wireless sensor networks, and cognitive radio networks. He is an Editor of *ACM Computing Surveys* and was an Associate Editor of IEEE TRANSACTIONS ON VEHICULAR TECHNOLOGY.